4.2 Interrupt Programming

Why Interrupt?

- A computer is much more than the CPU
  - Keyboard, mouse, screen, disk drives, scanner, printer, sound card, camera, etc.
- These devices occasionally need CPU service
  - But we can't predict when
- Solution 1: CPU periodically checks each device to see if it needs service – Polling

Disadvantage of Polling

- “Polling is like picking up your phone every few seconds to see if you have a call.”
- You may get a few phone calls a day → 99% of your effort will be wasted checking the phone.
- Takes CPU time even when no request pending

Example of Polling in Timer

This program toggles the rightmost LED every 1 second:

```
1  // Constants
2  .equ PRIVATE_TIMER_BASE, 0xFFFEC600 // Private Timer Base
3  .equ LED_BASE, 0xFF200000 // LED Base
4
5  .org 0x1000
6  .text
7  .global _start
8  _start:
9
10 ldr r0, =LED_BASE
11 mov r2, #0
12 str r2, [r0]
13
14 ldr r1, =PRIVATE_TIMER_BASE // MPCore private timer base address
15 mov r4, #1
16 str r4, [r1, #0xC] //Writing 1 to the interrupt status bit resets it
17
18 ldr r3, =200000000 // timeout = 1/(200 MHz) x 200 x 10^6 = 1 sec
19 str r3, [r1] // write to timer load register
20 mov r3, #0b011 // set bits: mode = 1 (auto), enable = 1
21 str r3, [r1, #0x8] // write to timer control register
22 LOOP:
23 WAIT: ldr r3, [r1, #0xC] // read timer status
24 cmp r3, #0
25 beq WAIT // wait for timer to expire
26 str r4, [r1, #0xC] // reset timer flag bit
27 eor r2, r2, #1 // toggle LEDG value
28 str r2, [r0]
29 b LOOP
30 .end                  // write all digits to seven segment display
```

Address	31 ··· 16	15 ··· 8	7 3	2	1	0	Register name
0xFFFEC600	Load value						Load
0xFFFEC604	Current value						Counter
0xFFFEC608	Unused	Prescaler	Unused	I	A	E	Control
0xFFFEC60C	Unused					F	Interrupt status

Interrupt

- Whenever a device needs the CPU's service, the device notifies it by sending an interrupt signal.
- CPU then stops and serves the device.
- "Polling is like picking up your phone every few seconds to see if you have a call. Interrupts are like waiting for the phone to ring."

Response to Interrupts

- If the interrupt request is present, the processor:
  - Completes the execution of the current instruction
  - Saves the address of the program counter on the link register so that the processor knows where to return to after handling the interrupt.
  - is redirected to the memory location where the interrupt request can be met. The location is listed in a *vector table*.
- The set of instructions written to meet the request is called an interrupt service routine (ISR). *At the end of the ISR, there must be an indication that the interrupt has been served (so that it would not be served repeatedly).*
- Once the request is accomplished, the processor should find its way back to the next instruction, where it was interrupted.

ARM Operating Modes

- ARM has seven basic operating modes. The following modes are related to interrupts:
  - Supervisor (SVC): entered when the processor is first powered on
  - IRQ: entered when a normal interrupt is raised
  - FIQ: entered when a high-priority interrupt is raised.
- We just consider IRQ interrupt in this course.

CPSR

- The operating mode is indicated in the CPSR:

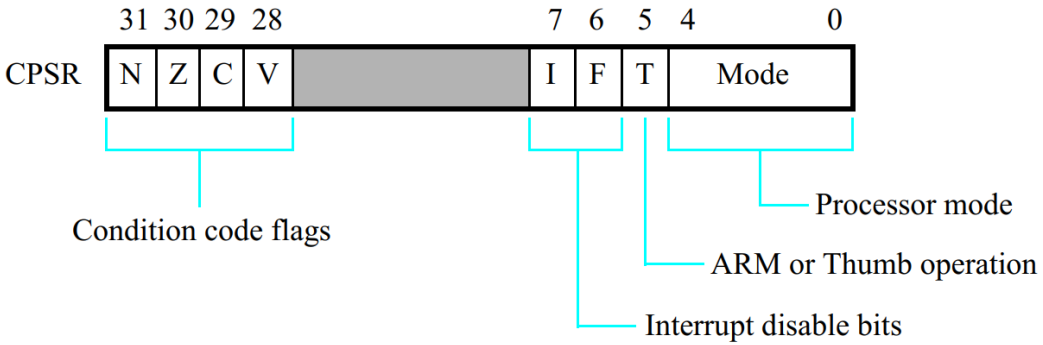
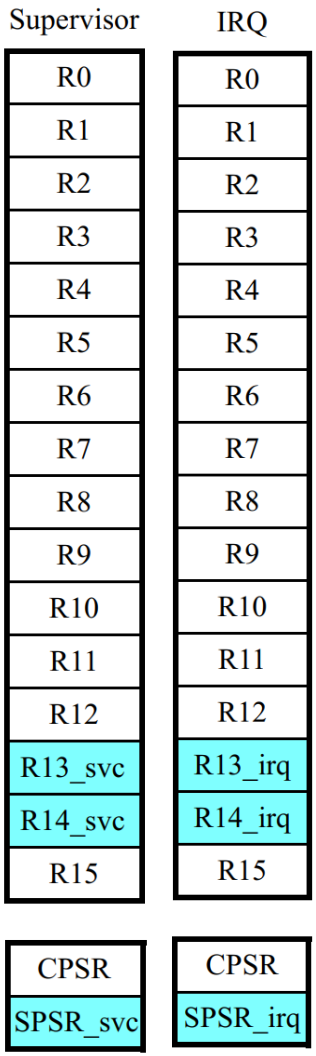


TABLE 1. Mode Bits

$CPSR_{4-0}$	Operating Mode
10000	User
10001	FIQ
10010	IRQ
10011	Supervisor
10111	Abort
11011	Undefined
11111	System

ARM Register Set

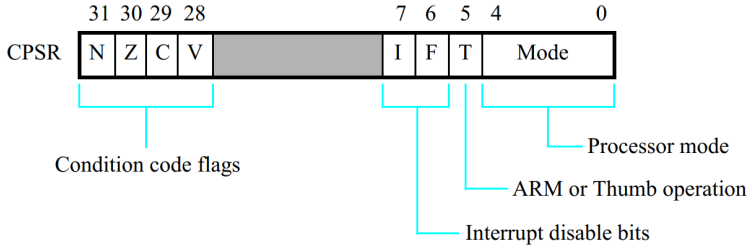


- r0-r12, r15 (pc) are common to Supervisor and IRQ modes.
- r13 (sp) and r14 (lr) are unique in each mode.
- CPSR is common to all modes
- SPSR is for preservation and restoration of CPSR, as discussed next.

Entering the interrupt handling workflow

If the interrupt request is present, the processor:

- Completes the execution of the current instruction
- Switches from SVC to IRQ mode.
  - Saves the current pc to the link register of the IRQ mode.
  - Copies CPSR to the SPSR in the IRQ mode.
  - IRQ disable bits set to 1: When handling an interrupt, another interrupt would not be responded to.



- is redirected to the memory location where the interrupt request can be met. The location is listed in a *vector table:*
  - IRQ – 0x00000018

Leaving the interrupt handling workflow

After the interrupt is handled, the processor:

- Copies SPSR back to CPSR, which will:
  - automatically switch back from the IRQ to the SVC mode.
  - Clear the IRQ disable bit, so that the processor can handle other interrupts.
- Move the link register back (minus an offset, will be discussed) to pc to return to where interrupt occurred.

Link register offset

- Recall that due to pipelining, when an instruction executes, the current pc points to 8 bytes in advance of the instruction.
- This is stored in the lr of the IRQ mode
- The current instruction will complete execution before entering the interrupt handling workflow.
- The next instruction that the processor should return to have an address of lr-4.

Line 1: instruction being executed when interrupt is triggered

Line 2: Want to return to this address after interrupt

Line 3: ← pc points to here, lr of the IRQ mode stores this address.

Link register offset

- Therefore, the ISR is returned to the main function by using the instruction:

```
subs pc, lr, #4
```

- This instruction:
  - Move the pc back to where the interrupt occurred.
  - Copies SPSR back to CPSR, which will:
    - automatically switch back from the IRQ to the SVC mode.
    - Clear the IRQ disable bit, so that the processor can handle other interrupts.

🧠 Micro-challenge

At the time of interrupt, LR contains:

A. Address of current instruction
B. Address of previous instruction
C. Address 8 bytes ahead of current instruction
D. Address of ISR

🧠 Micro-challenge

When returning from an interrupt service routine (ISR), why is the instruction

```
SUBS PC, LR, #4
```

used instead of simply moving LR into PC?

A. To skip the instruction that caused the interrupt
B. To compensate for the ARM pipeline so execution resumes at the correct next instruction
C. To restore general-purpose registers before returning
D. To disable further interrupts before resuming execution

🧠 Micro-challenge

Besides updating PC, what else does `subs pc, lr, #4` do?

A. Clears general registers
B. Restores CPSR from SPSR
C. Disables interrupts permanently
D. Flushes cache

Generic Interrupt Controller (GIC)

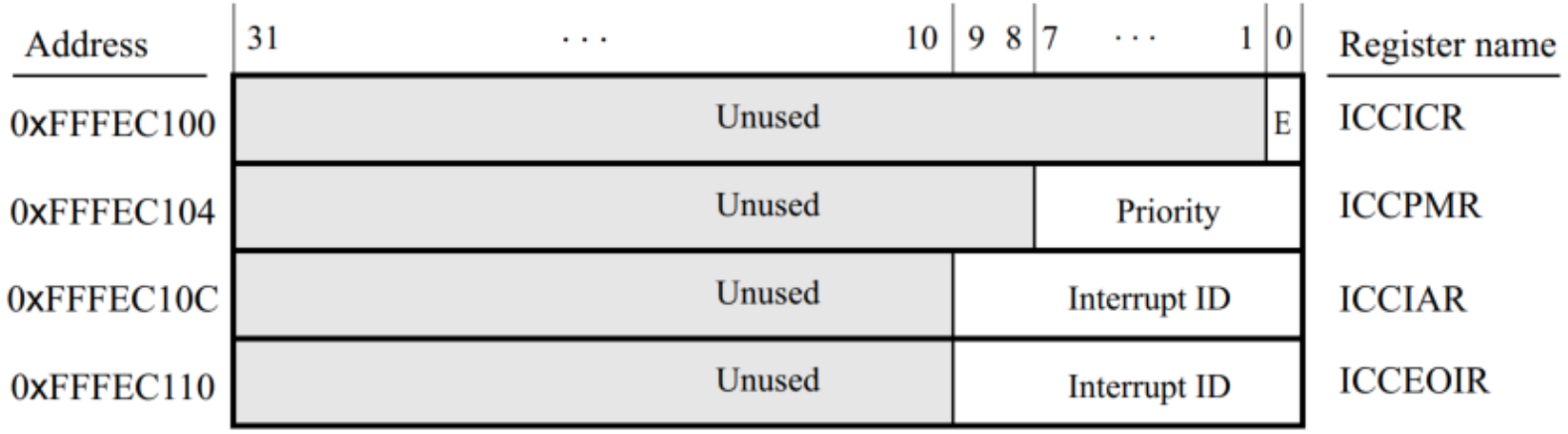
- Interrupts associated with I/O peripherals are handled by GIC.
- The GIC activates IRQ interrupt in the processor.
- Each I/O peripheral is identified in the GIC by an interrupt ID number. The interrupt IDs of some devices in CPUlator are listed below:

I/O Peripheral	Interrupt ID
Cortex-A9 Private timer	29
Cortex-A9 Watchdog timer	30
Interval timer 1	72
Push buttons	73
Interval timer 2	74
JTAG UART	80
HPS watchdog timer 1	203
HPS watchdog timer 2	204

Generic Interrupt Controller (GIC)

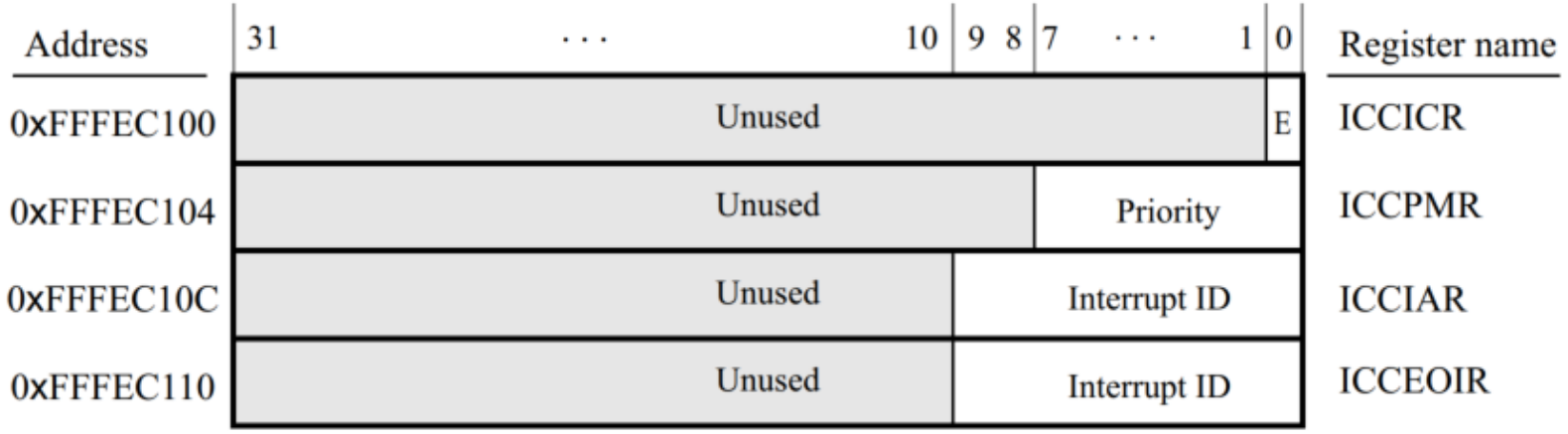
- The GIC consists of the distributor and the CPU Interface
- The distributor receives IRQ interrupt signals from I/O peripherals.
- The CPU interface is responsible for sending IRQ requests to the processor.
- When programming interrupt, we need to ensure both are enabled.
- Key registers needed to be set are listed below.

CPU interface registers



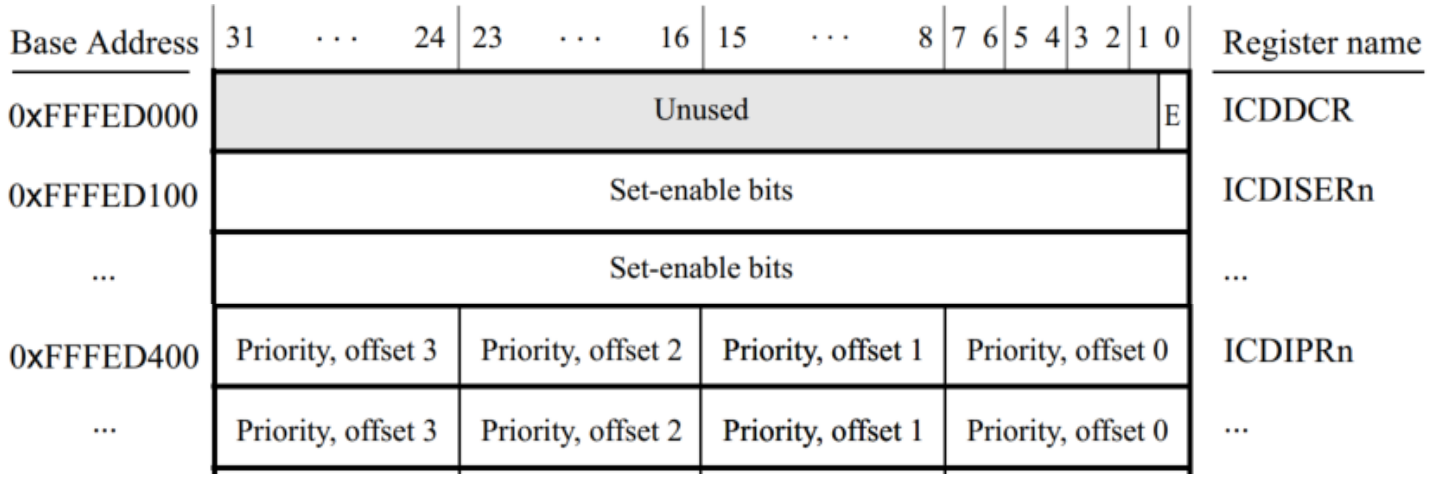
- CPU Interface Control Register (ICCICR): The E bit enables sending of the interrupt
- Interrupt Priority Mask Register (ICCPMR): set a threshold for the priority-level of the interrupts that will be served.
  - 0 is the highest priority interrupt
  - 255 is the lowest priority interrupt
  - ICCPMR = 0: No interrupt will be served
  - ICCPMR = 255: Interrupts with priority higher than 255 will be served

CPU interface registers



- Interrupt Acknowledge Register (ICCIAR)
  - contains the Interrupt ID of the I/O peripheral that has caused an interrupt.
  - ISR must read ICCIAR to determine which I/O peripheral has caused the interrupt.
- End of Interrupt Register (ICCEOIR)
  - Writing the interrupt ID to ICCEOIR indicates that the interrupt has been served.

GIC distributor registers



Base Address	31 ··· 24	23 ··· 16	15 ··· 8	7 6	5 4	3 2	1 0	Register name
0xFFFED000	Unused						E	ICDDCR
0xFFFED100	Set-enable bits							ICDISERn
...	Set-enable bits							...
0xFFFED400	Priority, offset 3	Priority, offset 2	Priority, offset 1		Priority, offset 0			ICDIPRn
...	Priority, offset 3	Priority, offset 2	Priority, offset 1		Priority, offset 0			...

- Distributor control register (ICDDCR)
  - Setting E = 1 enables the distributor
- Interrupt Set Enable Registers (ICDISERn)
  - Enable/disable I/O peripherals with Interrupt IDs
  - e.g., 0xFFFED100 enables/disables whether an interrupt is triggered for each of the I/O peripherals with interrupt IDs from 0 to 31.
- Interrupt Priority Registers (ICDIPRn)
  - Sets the priority level of each interrupt
  - e.g., 0xFFFED400 sets the priority level of each of the I/O peripherals with interrupt IDs from 0 to 3.

Putting it together …..

- The following provides an example program that displays:
  - 0 in the 7-segment LED when the first button is pushed.
  - 1 when the second button is pushed.
  - 2 when the third button is pushed
  - 3 when the fourth button is pushed
- The program is provided as `PushbuttonInterrupt.s`

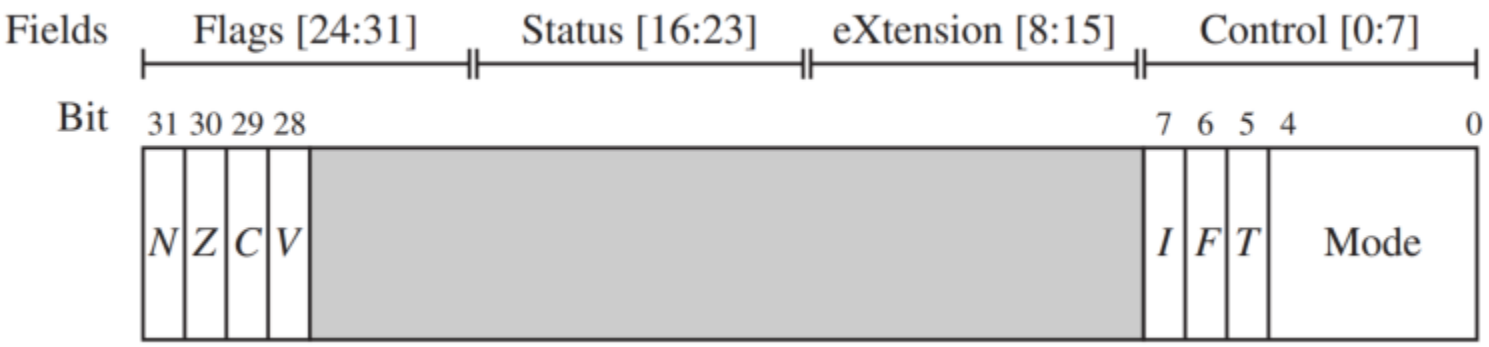
Initialization (1)

```
11 .text
12 .global _start
13 _start:
14     /* Set up stack pointers for IRQ and SVC processor modes */
15     MOV R1, #0b11010010 // interrupts masked, MODE = IRQ
16     MSR CPSR_c, R1 // change to IRQ mode
17     LDR SP, =0xFFFFFFFF - 3
18     /* Change to SVC (supervisor) mode with interrupts disabled */
19     MOV R1, #0b11010011 // interrupts masked, MODE = SVC
20     MSR CPSR_c, R1 // change to supervisor mode
21     LDR SP, =0x3FFFFFFF - 3
```

- The purpose of this code is to set up different stack pointers in the IRQ and SVC modes.
- The stack in IRQ and SVC modes are independent and should not interfere with each other.

Initialization (1)

- The msr instruction transfers the contents of a register into CPSR
- Note that the parameter of the msr instruction is CPSR_c.
- c is the Control field that refers to the least significant byte of CPSR, which controls the interrupt masks and the processor mode.



Initialization (2)

```
23      BL CONFIG_GIC // configure the ARM GIC
```

- The CONFIG_GIC subroutine:
  - enables the distributors and CPU interface
  - Set the priority thresholds (ICCPMR) to be 255 so that all interrupts are enabled.
  - Enable the Push button interrupt.

Initialization (3)

```
25    // write to the pushbutton KEY interrupt mask register
26    LDR R0, =0xFF200050 // pushbutton KEY base address
27    MOV R1, #0xF // set interrupt mask bits
28    STR R1, [R0, #0x8] // interrupt mask register (base + 8)
29
30    LDR R2, =#0xFFFFFFFF
31    STR R2, [R0, #0xC] // interrupt mask register (base + 12).
32    //Reset all the interrupt flag.
```

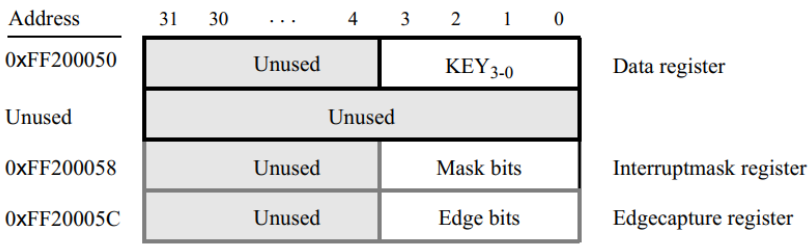
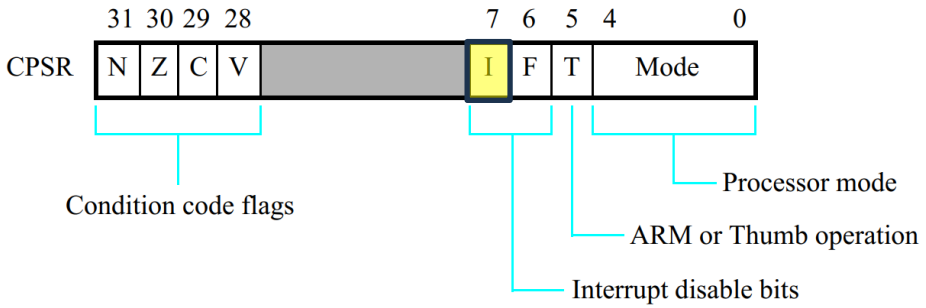


Figure 12. Registers used in the pushbutton parallel port.

- Enable interrupt for all four buttons by enabling all bits in the interrupt mask register.
- Each bit in the edge capture register has a value of 1 if the corresponding bit location has changed from 1 to 0.
- Writing to this register clears all values in this register.

Initialization (4)

```
34      // enable IRQ interrupts in the processor
35      MOV R0, #0b01010011 // IRQ unmasked, MODE = SVC
36      MSR CPSR_c, R0
37  IDLE:
38      B IDLE
```



- Clear the IRQ disable bit in CPSR (i.e., enable IRQ)
- Enter an infinite loop and wait for the push button interrupt to be triggered.

Micro-challenge

Given the interrupt program initialized by this code:

```
        MOV R0, #0b01010011
        MSR CPSR_c, R0
IDLE:
        B IDLE
```

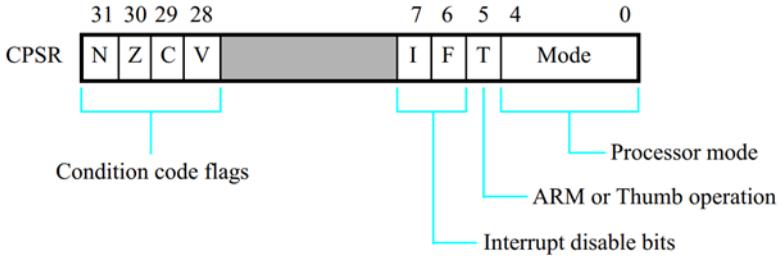


TABLE 1. Mode Bits

$CPSR_{4-0}$	Operating Mode
10000	User
10001	FIQ
10010	IRQ
10011	Supervisor
10111	Abort
11011	Undefined
11111	System

- What is the value of the control field of CPSR and the SPSR of the IRQ mode immediately after an interrupt is triggered?

Exception vector table

```
1 .section .vectors, "ax"
2 B _start // reset vector
3 B SERVICE_UND // undefined instruction vector
4 B SERVICE_SVC // software interrrupt vector
5 B SERVICE_ABT_INST // aborted prefetch vector
6 B SERVICE_ABT_DATA // aborted data vector
7 .word 0 // unused vector
8 B SERVICE_IRQ // IRQ interrupt vector
9 B SERVICE_FIQ // FIQ interrupt vector
```

- Different exceptions are directed to different addresses. The entries in the vector table above provide branches to various exception service routines.
  - e.g., when an IRQ is triggered, the pc will be redirected to 0x00000018 at which the instruction `b service_irq` will branch to the ISR handling the IRQ interrupt.

ISR

```
52 SERVICE_IRQ:
53     stmfd sp!, {R0-R7, LR}
54 /* Read the ICCIAR from the CPU Interface */
55     LDR R4, =0xFFFEC100
56     LDR R5, [R4, #0x0C] // read from ICCIAR
57 FPGA_IRQ1_HANDLER:
58     CMP R5, #73
59 UNEXPECTED:
60     BNE EXIT_IRQ // if not recognized, stop here
61     BL KEY_ISR
62 EXIT_IRQ:
63 /* Write to the End of Interrupt Register (ICCEOIR) */
64     STR R5, [R4, #0x10] // write to ICCEOIR
65     ldmfd sp!, {R0-R7, LR}
66     SUBS PC, LR, #4 // S flag set, and the PC as destination register:
67     //Updates the PC but also copies the SPSR into the CPSR
```

This comparison checked whether it is the push buttons triggering an interrupt.

This ensures that GIC knows that the interrupt has been served already.
If not done: The interrupt remains active in the GIC, preventing other interrupts from being serviced.

Restore CPSR:

- automatically switch back from the IRQ to the SVC mode.
- Clear the IRQ disable bit, so that the processor can handle other interrupts.

Micro-challenge

```
52 SERVICE_IRQ:
53     stmfd sp!, {R0-R7, LR}
54 /* Read the ICCIAR from the CPU Interface */
55     LDR R4, =0xFFFEC100
56     LDR R5, [R4, #0x0C] // read from ICCIAR
57 FPGA_IRQ1_HANDLER:
58     CMP R5, #73
59 UNEXPECTED:
60     BNE EXIT_IRQ // if not recognized, stop
61     BL KEY_ISR
62 EXIT_IRQ:
63 /* Write to the End of Interrupt Register (ICCEOIR) */
64     STR R5, [R4, #0x10] // write to ICCEOIR
65     ldmfd sp!, {R0-R7, LR}
66     SUBS PC, LR, #4 // S flag set, and the PC as destination register:
67     //Updates the PC but also copies the SPSR into the CPSR
```

Address	31 ··· 10	9	8	7 ··· 1	0	Register name
0xFFFEC100	Unused				E	ICCICR
0xFFFEC104	Unused			Priority		ICCPMR
0xFFFEC10C	Unused			Interrupt ID		ICCIAR
0xFFFEC110	Unused			Interrupt ID		ICCEOIR

1. What is the purpose of the instruction:

Line 58: `CMP R5, #73`

2. What is the purpose of:

Line 64: `STR R5, [R4, #0x10]`

ISR

```
KEY_ISR:
    stmfd sp!,{lr}
    LDR R0, =0xFF200050 // base address of pushbutton KEY port
    LDR R1, [R0, #0xC] // read edge capture register
    MOV R2, #0xF
    STR R2, [R0, #0xC] // clear the interrupt
    LDR R0, =0xFF200020 // based address of HEX display
CHECK_KEY0:
    MOV R3, #0x1
    ANDS R3, R3, R1 // check for KEY0
    BEQ CHECK_KEY1
    MOV R2, #0b00111111
    STR R2, [R0] // display "0"
    B END_KEY_ISR
CHECK_KEY1:
    MOV R3, #0x2
    ANDS R3, R3, R1 // check for KEY1
    BEQ CHECK_KEY2
    MOV R2, #0b00000110
    STR R2, [R0] // display "1"
    B END_KEY_ISR
CHECK_KEY2:
    MOV R3, #0x4
    ANDS R3, R3, R1 // check for KEY2
    BEQ IS_KEY3
    MOV R2, #0b01011011
    STR R2, [R0] // display "2"
    B END_KEY_ISR
IS_KEY3:
    MOV R2, #0b01001111
    STR R2, [R0] // display "3"
END_KEY_ISR:
    ldmfd sp!,{pc}
    //BX LR
.end
```

If zero, then button 0 is not pressed.

If zero, then button 1 is not pressed.

If zero, then button 2 is not pressed.

If not button 0, 1 or 2, then it must be button 3 that is pressed.

- Identify which button has been pressed by checking the edge capture register
- After reading, clear the register, to allow the process to start afresh for the next interrupt.

Address	31 30 · · · 4	3 2 1 0	
0xFF200050	Unused	KEY_{3-0}	Data register
Unused	Unused		
0xFF200058	Unused	Mask bits	Interruptmask register
0xFF20005C	Unused	Edge bits	Edgecapture register

Figure 12. Registers used in the pushbutton parallel port.